

TSAR Project Part 3 - Times Series Forecasting

Kryštof Vávra, Said Fakhri (Group 03)

2026-06-01

1 Load Data

1.1 Data Loaded

```
mySeries <- readRDS("employment_series_CEU1000000001.rds")

#Quick summary
summary(mySeries)
```

Month	Series_ID	Title	Employed
Min. :1939 Jan	Length:969	Length:969	Min. : 556.0
1st Qu.:1959 Mar	Class :character	Class :character	1st Qu.: 676.0
Median :1979 May	Mode :character	Mode :character	Median : 765.0
Mean :1979 May			Mean : 793.8
3rd Qu.:1999 Jul			3rd Qu.: 899.0
Max. :2019 Sep			Max. :1260.0

```
#Check for NAs
sum(is.na(mySeries))
```

```
[1] 0
```

2 Train 2 different benchmark models

2.1 The seasonal naive method (see Part 2, Lecture 3)

2.1.1 Extract Training Set

The number of rows in the time series is counted using `nrow()`. Then 80% of the rows are calculated and rounded down with `floor()`. The first 80% of the observations are extracted with `slice()` and stored as the training set in the variable `train`. The remaining 20% are not extracted separately, but their length is stored for later forecasting.

```
# Count total number of rows
n_total <- nrow(mySeries)

# Calculate the number of rows for the training set
n_train <- floor(n_total * 0.8)

# Calculate the length of the remaining test period
test_length <- n_total - n_train

# Extract the first 80% of rows as the training set
train <- mySeries %>%
  slice(1:n_train)

# Show the calculated values
n_total
```

```
[1] 969
```

```
n_train
```

```
[1] 775
```

```
test_length
```

```
[1] 194
```

```
# Inspect the training set
train
```

```
# A tsibble: 775 x 4 [1M]
# Key:      Series_ID [1]
  Month Series_ID Title      Employed
  <mtch> <chr>    <chr>      <dbl>
1 1939 Jan CEU1000000001 Mining and Logging 885
2 1939 Feb CEU1000000001 Mining and Logging 882
3 1939 Mar CEU1000000001 Mining and Logging 883
4 1939 Apr CEU1000000001 Mining and Logging 614
5 1939 May CEU1000000001 Mining and Logging 719
6 1939 Jun CEU1000000001 Mining and Logging 852
7 1939 Jul CEU1000000001 Mining and Logging 847
8 1939 Aug CEU1000000001 Mining and Logging 864
9 1939 Sep CEU1000000001 Mining and Logging 894
10 1939 Oct CEU1000000001 Mining and Logging 941
# i 765 more rows
```

2.1.2 Train your 1st Benchmark Model

The first benchmark model is the seasonal naive method. This model is useful for monthly time series because it forecasts each future value using the value from the same month in the previous year. The model is trained on the training set using `model()`. The function `augment()` is then used to inspect the fitted values and residuals.

```
# Train the first benchmark model: seasonal naive
model_snaive <- train %>%
  model(
    seasonal_naive = SNAIVE(Employed)
  )

# Show the trained model
model_snaive
```

```
# A mable: 1 x 2
# Key:      Series_ID [1]
  Series_ID      seasonal_naive
  <chr>          <model>
1 CEU1000000001 <SNAIVE>
```

2.1.3 Inspect Fitted Values of the 1st Benchmark Model

The fitted values are inspected using `augment()`. The first fitted values may be missing because the seasonal naive method needs observations from the previous season before it can produce fitted values.

```
# Get fitted values and residuals
augmented_snaive <- augment(model_snaive)

# Inspect fitted values
augmented_snaive
```

```
# A tsibble: 775 x 7 [1M]
# Key:   Series_ID, .model [1]
  Series_ID      .model      Month Employed .fitted .resid .innov
  <chr>         <chr>      <mt>   <dbl>   <dbl>   <dbl>   <dbl>
1 CEU1000000001 seasonal_naive 1939 Jan     885     NA     NA     NA
2 CEU1000000001 seasonal_naive 1939 Feb     882     NA     NA     NA
3 CEU1000000001 seasonal_naive 1939 Mar     883     NA     NA     NA
4 CEU1000000001 seasonal_naive 1939 Apr     614     NA     NA     NA
5 CEU1000000001 seasonal_naive 1939 May     719     NA     NA     NA
6 CEU1000000001 seasonal_naive 1939 Jun     852     NA     NA     NA
7 CEU1000000001 seasonal_naive 1939 Jul     847     NA     NA     NA
8 CEU1000000001 seasonal_naive 1939 Aug     864     NA     NA     NA
9 CEU1000000001 seasonal_naive 1939 Sep     894     NA     NA     NA
10 CEU1000000001 seasonal_naive 1939 Oct     941     NA     NA     NA
# i 765 more rows
```

```
View(augmented_snaive)
```

2.1.4 Evaluate Model Fit of 1st Benchmark Model: Actual vs. Fitted Time Plot

The model fit is first evaluated by comparing the actual values of the training set with the fitted values produced by the seasonal naive benchmark model.

```
# Get fitted values and residuals from the seasonal naive model
augmented_snaive <- augment(model_snaive)

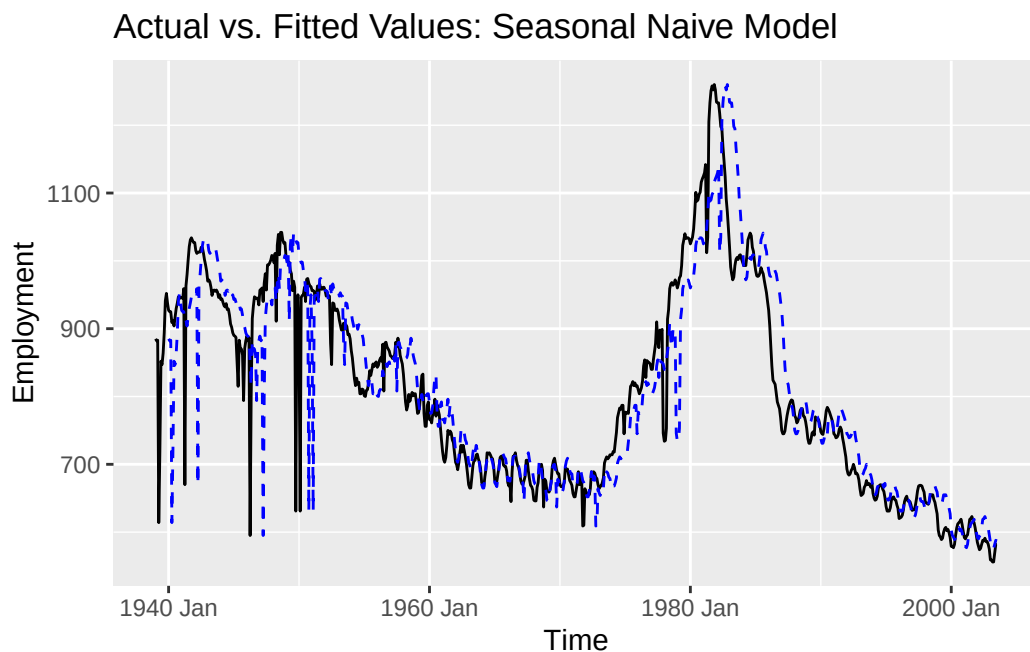
# Plot actual values and fitted values in the same time plot
augmented_snaive %>%
  autoplot(Employed) +
```

```

autolayer(augmented_snaive, .fitted, color = "blue", linetype = "dashed") +
labs(
  title = "Actual vs. Fitted Values: Seasonal Naive Model",
  x = "Time",
  y = "Employment"
)

```

Warning: Removed 12 rows containing missing values or values outside the scale range (`geom\_line()`).



Are the fitted values close to the actual values?

The fitted values follow the general movement of the actual values, but they do not match them perfectly.

Are they systematically too high or too low?

The fitted values are systematically too low during increasing periods and too high during decreasing periods.

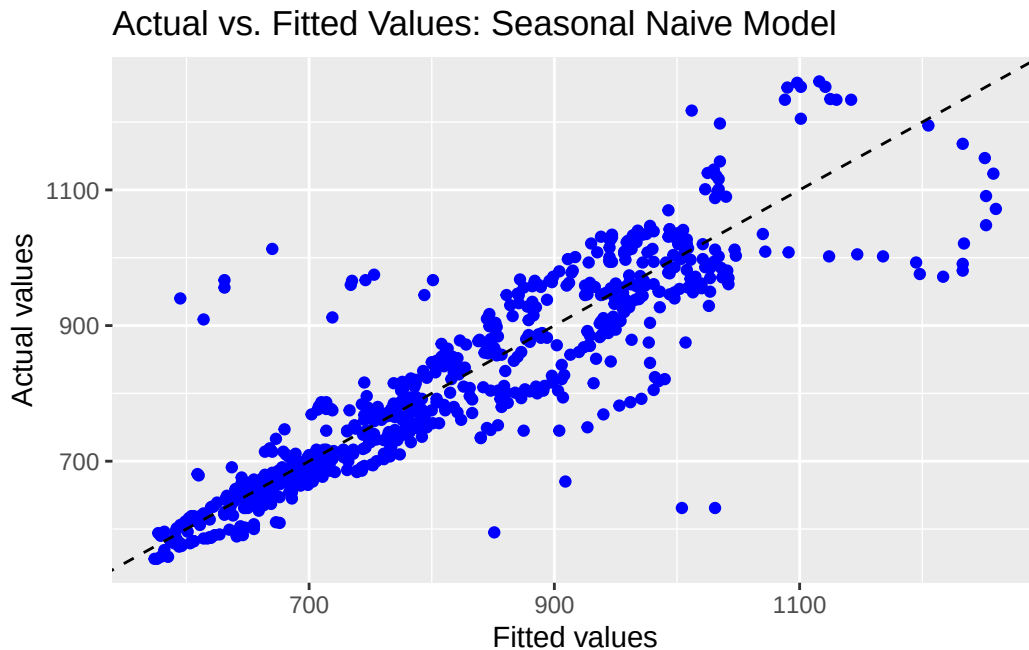
If yes, where does it come from?

This happens because the seasonal naive method uses the value from the same month in the previous year, so it reacts with a one-year delay. (In documentation it is stated that without lag specification the seasonal interval is 1 year for monthly data.)

2.1.5 Actual vs. Fitted Scatter Plot

The actual values are plotted against the fitted values to check how close the fitted values are to the actual observations. The dashed identity line represents perfect predictions, where fitted values would be exactly equal to actual values.

```
augmented_snaive %>%  
  filter(!is.na(.fitted)) %>%  
  ggplot(aes(x = .fitted, y = Employed)) +  
  geom_point(color = "blue") +  
  geom_abline(slope = 1, intercept = 0, linetype = 2) +  
  labs(  
    title = "Actual vs. Fitted Values: Seasonal Naive Model",  
    x = "Fitted values",  
    y = "Actual values"  
  )
```



Are the points close to the identity line?

Many points are close to the identity line, especially at lower employment levels, but the spread becomes much larger at higher employment levels.

Are they systematically too high or too low?

There is no constant overestimation or underestimation across the whole plot, but the model shows larger errors during periods where the employment level changes more.

If yes, where does it come from?

The larger errors occur because the seasonal naive model uses values from the same month in the previous year, so it reacts with a one-year delay when employment changes suddenly.

2.1.6 Residual Diagnostics

The residuals of the seasonal naive model are inspected using `gg_tsresiduals()`. A good forecasting model should have residuals with no clear autocorrelation and a mean close to zero.

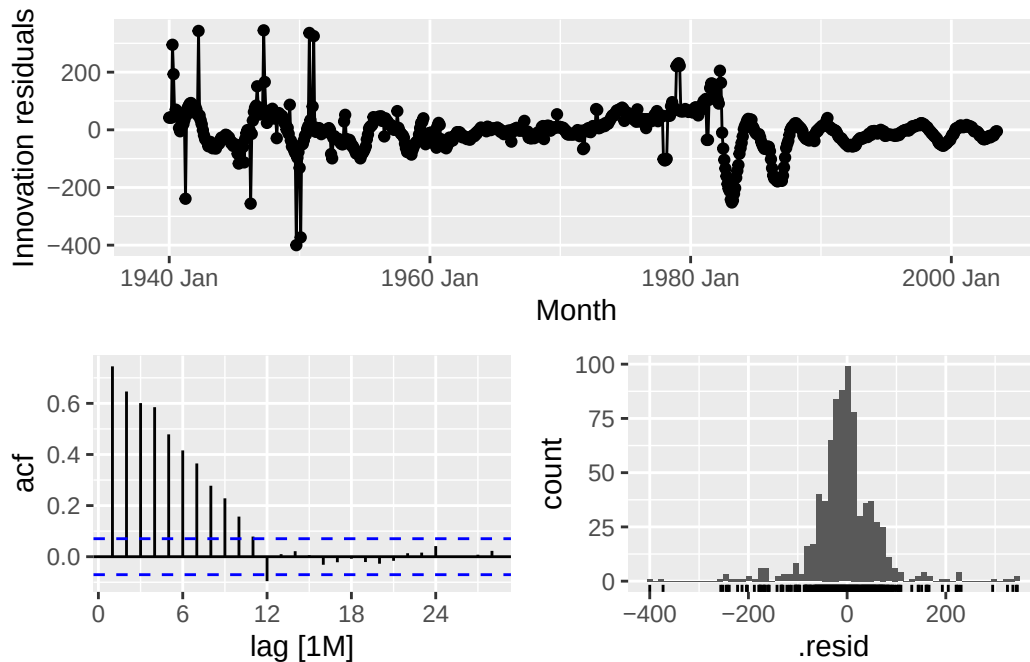
```
model_snaive %>%  
  gg_tsresiduals()
```

Warning: Removed 12 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 12 rows containing missing values or values outside the scale range (``geom_point()``).

Warning: Removed 12 rows containing non-finite outside the scale range (``stat_bin()``).

Warning: Removed 12 rows containing missing values or values outside the scale range (``geom_rug()``).



Are the residuals auto-correlated? How do you decide that based on the plots?

Yes, the residuals are auto-correlated because the ACF plot shows several significant spikes outside the blue significance bounds. The model does not capture the short term dependence in the data. (all outside the significance line is lag < 12)

Do the residuals have zero mean? How do you decide that based on the plots?

The residuals appear to be roughly centered around zero because the histogram is concentrated around zero and the residual time plot fluctuates around zero, although some outliers are visible and that could influence the mean. We cannot be 100% sure just from the plot.

What do these results tell you about your model?

The seasonal naive model captures part of the yearly seasonal pattern, but the autocorrelation in the residuals shows that important short-term patterns remain unexplained.

2.1.7 Double-check Residual Diagnostics

The residual diagnostics are double-checked using the Ljung-Box test for residual autocorrelation and by calculating the mean of the residuals. Since the data is monthly and the seasonal period is 12 months, a lag of 24 is used for the Ljung-Box test.

```
# Ljung-Box test for residual autocorrelation
lb_snaive <- augmented_snaive %>%
  features(.resid, ljung_box, lag = 24, dof = 0)
```



```
lb_snaive
```

```
# A tibble: 1 x 4
  Series_ID      .model      lb_stat lb_pvalue
  <chr>         <chr>         <dbl>   <dbl>
1 CEU1000000001 seasonal_naive  1834.     0
```

```
# Calculate residual mean
residual_mean_snaive <- mean(augmented_snaive$.resid, na.rm = TRUE)

residual_mean_snaive
```

```
[1] -4.440367
```

Does the Ljung-Box test support your conclusions from the residual plots? How do you conclude that from the test result?

Yes, the Ljung-Box test supports the conclusion that the residuals are autocorrelated, because the p-value is effectively 0, which is below 0.05, so the null hypothesis of no residual autocorrelation is rejected.

Does the residual mean support your conclusions from the residual plots?

Yes, the residual mean is -4.44, which is very close to zero compared with the employment values, so it supports the conclusion that the residuals are roughly centered around zero.

2.1.8 Evaluate Point Forecast Accuracy of 1st Benchmark Model

The trained seasonal naive model is used to forecast the remaining 20% of the time series. The forecast horizon is set to `test_length`, which is the number of observations not included in the training set.

```
# Create forecast for the same length as the test period
fc_snaive <- model_snaive %>%
  forecast(h = test_length)

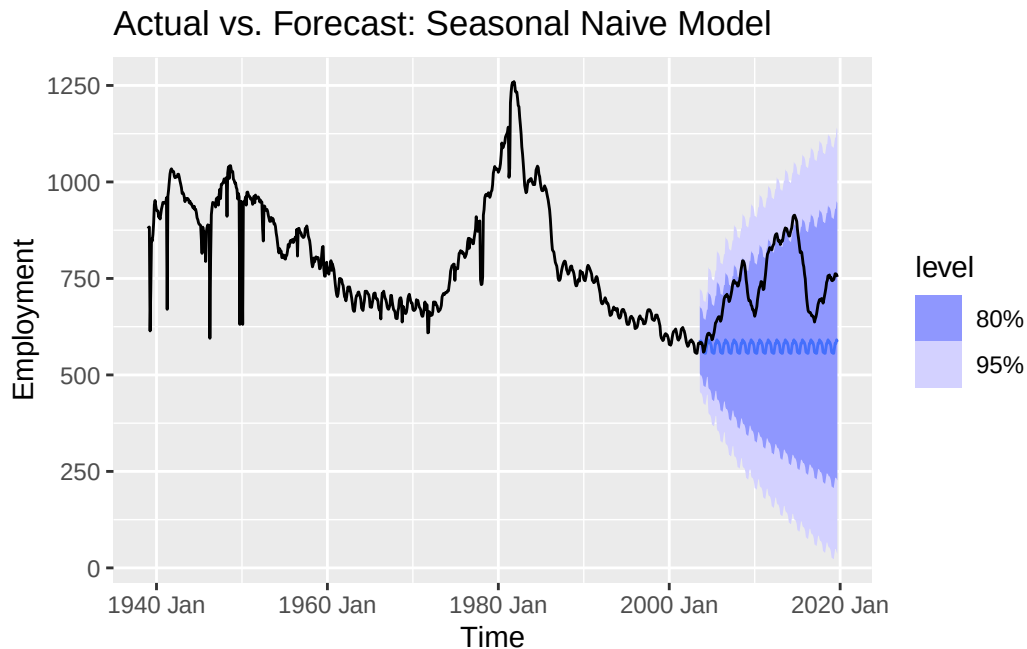
# Inspect forecast
fc_snaive
```

```
# A fable: 194 x 5 [1M]
# Key:      Series_ID, .model [1]
  Series_ID      .model      Month
  <chr>          <chr>      <mth>
1 CEU1000000001 seasonal_naive 2003 Aug
2 CEU1000000001 seasonal_naive 2003 Sep
3 CEU1000000001 seasonal_naive 2003 Oct
4 CEU1000000001 seasonal_naive 2003 Nov
5 CEU1000000001 seasonal_naive 2003 Dec
6 CEU1000000001 seasonal_naive 2004 Jan
7 CEU1000000001 seasonal_naive 2004 Feb
8 CEU1000000001 seasonal_naive 2004 Mar
9 CEU1000000001 seasonal_naive 2004 Apr
10 CEU1000000001 seasonal_naive 2004 May
# i 184 more rows
# i 2 more variables: Employed <dist>, .mean <dbl>
```

2.1.9 Actual vs. Forecast Time Plot

The forecast values are plotted together with the original time series to visually compare the predicted values with the actual observations in the test period.

```
fc_snaive %>%
  autoplot(mySeries) +
  labs(
    title = "Actual vs. Forecast: Seasonal Naive Model",
    x = "Time",
    y = "Employment"
  )
```



2.1.10 Forecast Accuracy Metrics

The point forecast accuracy is evaluated using accuracy metrics. The two main metrics interpreted here are RMSE and MAPE.

```
accuracy_snaive <- fc_snaive %>%
  accuracy(mySeries)

accuracy_snaive
```

```
# A tibble: 1 x 11
  .model      Series_ID .type    ME  RMSE  MAE  MPE  MAPE  MASE  RMSSE  ACF1
  <chr>      <chr>      <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 seasonal_naive CEU10000~ Test   161.  184.  161.  20.8  20.8  3.69  2.70  0.987
```

Actual vs. Forecast interpretation:

The seasonal naive forecast does not capture the strong increase in the test period. The forecast mostly repeats the seasonal pattern from the end of the training data, but the real employment values increase much more. This means that the model works as a simple benchmark, but it struggles when the series changes strongly and the future trend is different from the past pattern.

RMSE interpretation:

The RMSE of the seasonal naive model is 183.68. Since RMSE is scale-dependent, it must be interpreted relative to the employment level of the series. Looking at the time plot, this error is large because the model underestimates the actual values in the test period.

MAPE interpretation:

The MAPE of the seasonal naive model is 20.78%. This means that, on average, the forecast error is about 20.78% of the actual employment level. This is quite high for a point forecast and confirms that the seasonal naive model is not very accurate for this test period.

2.1.11 Evaluate Point Forecast Uncertainty of 1st Benchmark Model

The forecast uncertainty is evaluated by checking whether the residuals satisfy the assumptions needed for reliable prediction intervals. The same residual diagnostics as before are used, but the focus is now on constant variance and normality.

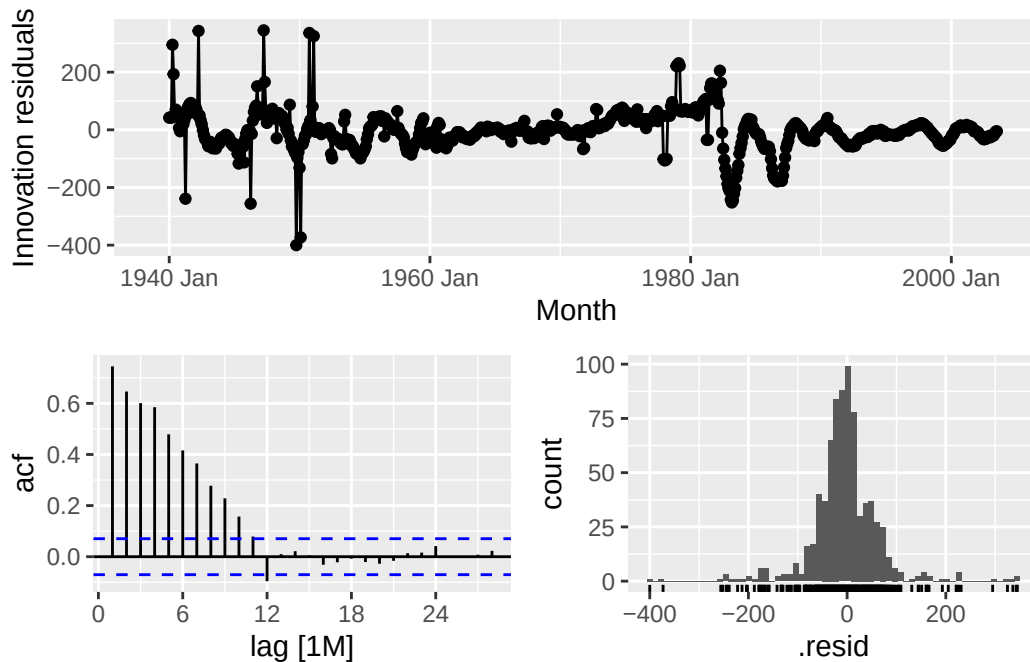
```
model_snaive %>%  
  gg_tsresiduals()
```

```
Warning: Removed 12 rows containing missing values or values outside the scale range  
(`geom_line()`).
```

```
Warning: Removed 12 rows containing missing values or values outside the scale range  
(`geom_point()`).
```

```
Warning: Removed 12 rows containing non-finite outside the scale range  
(`stat_bin()`).
```

```
Warning: Removed 12 rows containing missing values or values outside the scale range  
(`geom_rug()`).
```



Are the residuals homoscedastic?

No, the residuals do not appear to have constant variance because some periods show much larger residual fluctuations than others.

Are the residuals normally distributed?

No, the residuals are not perfectly normally distributed. Yes it is mostly symmetric and bell-shaped around zero, but it has big tails with distinct outliers. Especially around -400 and +300. So it approximates normality around 0 I need to say no because of the outliers.

2.1.12 Prediction Interval Boundaries

The prediction interval boundaries are displayed using `hilo()`. This shows the lower and upper bounds of the 80% and 95% prediction intervals for each forecasted month.

```
fc_snaive %>%
  hilo(level = c(80, 95)) %>%
  select(Month, .mean, `80%`, `95%`)
```

A tsibble: 194 x 4 [1M]

	Month	.mean	`80%`	`95%`
	<mt>	<dbl>	<hilo>	<hilo>
1	2003 Aug	591	[503.8657, 678.1343]	[457.7395, 724.2605]
2	2003 Sep	586	[498.8657, 673.1343]	[452.7395, 719.2605]

```

3 2003 Oct    586 [498.8657, 673.1343]80 [452.7395, 719.2605]95
4 2003 Nov    582 [494.8657, 669.1343]80 [448.7395, 715.2605]95
5 2003 Dec    575 [487.8657, 662.1343]80 [441.7395, 708.2605]95
6 2004 Jan    559 [471.8657, 646.1343]80 [425.7395, 692.2605]95
7 2004 Feb    559 [471.8657, 646.1343]80 [425.7395, 692.2605]95
8 2004 Mar    556 [468.8657, 643.1343]80 [422.7395, 689.2605]95
9 2004 Apr    556 [468.8657, 643.1343]80 [422.7395, 689.2605]95
10 2004 May   569 [481.8657, 656.1343]80 [435.7395, 702.2605]95
# i 184 more rows

```

Does it make sense to include these prediction intervals in the model evaluation?

The prediction intervals should only be interpreted with caution because the residuals are autocorrelated, not clearly homoscedastic, and not normally distributed, so the intervals are not that reliable.

2.2 Second Model: The Drift Method

2.2.1 Train your 2nd Benchmark Model

The second benchmark model is the drift method. This model is useful because it captures the long-term trend of a time series. The model is trained on the training set using `model()`. The function `augment()` is then used to inspect the fitted values and residuals.

```

# Train the 2nd benchmark model: drift method
model_drift <- train %>%
  model(
    drift = RW(Employed ~ drift())
  )

# Show the trained model
model_drift

```

```

# A mable: 1 x 2
# Key:      Series_ID [1]
  Series_ID      drift
  <chr>          <model>
1 CEU1000000001 <RW w/ drift>

```

2.2.2 Inspect Fitted Values of the 2nd Benchmark Model

The fitted values are inspected using `augment()`. The first fitted value is missing (NA) because the drift method produces 1-step forecasts, so there is no fitted value for the very first observation.

```
# Get fitted values and residuals
augmented_drift <- augment(model_drift)
```

```
# Inspect fitted values
augmented_drift
```

```
# A tsibble: 775 x 7 [1M]
# Key:   Series_ID, .model [1]
  Series_ID      .model   Month Employed .fitted .resid .innov
  <chr>         <chr>    <mth>   <dbl>   <dbl>   <dbl>   <dbl>
1 CEU1000000001 drift   1939 Jan     885     NA     NA     NA
2 CEU1000000001 drift   1939 Feb     882    885.   -2.61  -2.61
3 CEU1000000001 drift   1939 Mar     883    882.    1.39   1.39
4 CEU1000000001 drift   1939 Apr     614    883.  -269.  -269.
5 CEU1000000001 drift   1939 May     719    614.   105.   105.
6 CEU1000000001 drift   1939 Jun     852    719.   133.   133.
7 CEU1000000001 drift   1939 Jul     847    852.   -4.61  -4.61
8 CEU1000000001 drift   1939 Aug     864    847.    17.4   17.4
9 CEU1000000001 drift   1939 Sep     894    864.    30.4   30.4
10 CEU1000000001 drift   1939 Oct     941    894.    47.4   47.4
# i 765 more rows
```

```
View(augmented_drift)
```

2.2.3 Evaluate Model Fit of 2nd Benchmark Model: Actual vs. Fitted Time Plot

The model fit is first evaluated by comparing the actual values of the training set with the fitted values produced by the drift benchmark model.

```
# Get fitted values and residuals from the seasonal naive model
augmented_drift <- augment(model_drift)
```

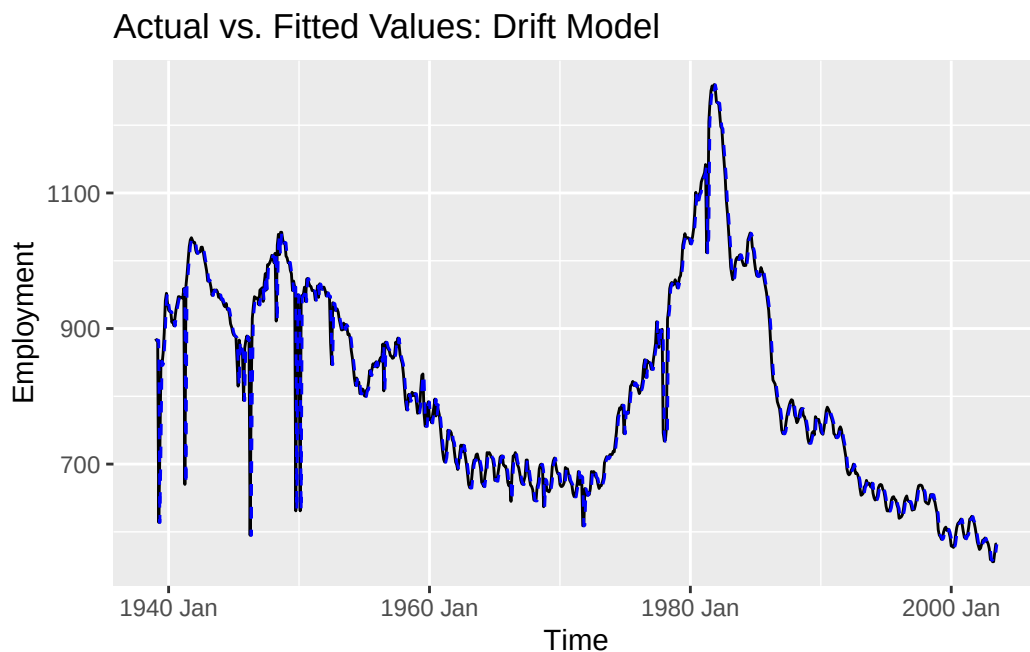
```
# Plot actual values and fitted values in the same time plot
augmented_drift %>%
  autoplot(Employed) +
```

```

autolayer(augmented_drift, .fitted, color = "blue", linetype = "dashed") +
labs(
  title = "Actual vs. Fitted Values: Drift Model",
  x = "Time",
  y = "Employment"
)

```

Warning: Removed 1 row containing missing values or values outside the scale range (`geom\_line()`).



Are the fitted values close to the actual values?

Yes. The fitted values are very close to the actual values and follow their movement almost exactly.

Are they systematically too high or too low?

No, there is no clear systematic bias. Each fitted value is essentially the previous observation shifted by the small drift. Because the drift coefficient is tiny relative to the level of the series, the fitted line lies almost on top of the actual series.

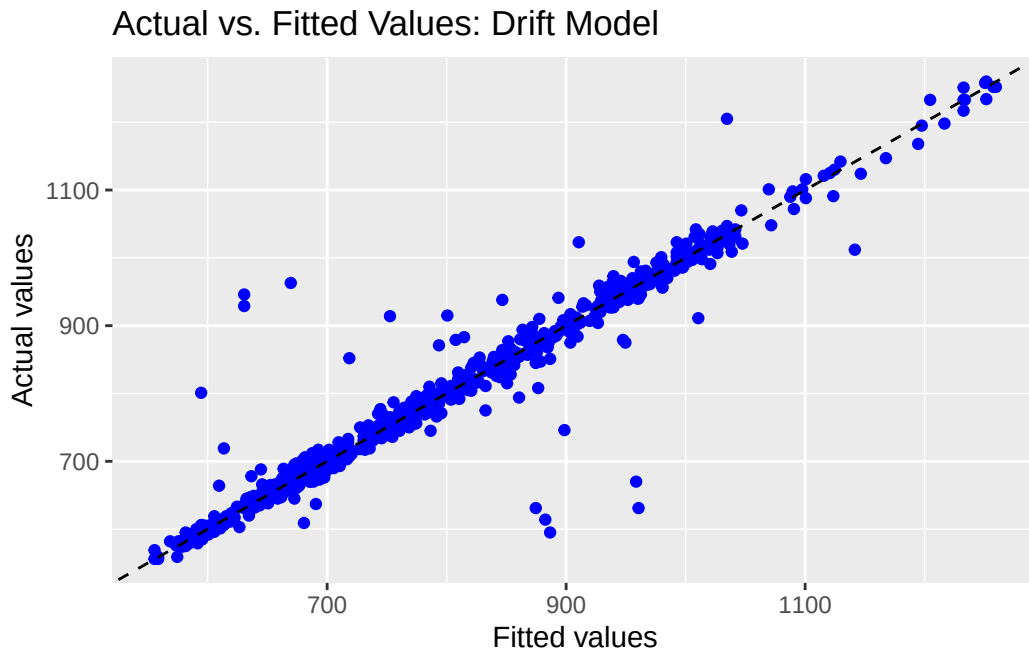
If yes, where does it come from?

Not applicable: There is no systematic over or underestimation. The fitted values track the actual values closely. If there is an abrupt change, the fitted have small delay because they cannot anticipate this abrupt change.

2.2.4 Actual vs. Fitted Scatter Plot

The actual values are plotted against the fitted values to check how close the fitted values are to the actual observations. The blue dashed identity line represents perfect predictions, where fitted values would be exactly equal to actual values.

```
augmented_drift %>%  
  filter(!is.na(.fitted)) %>%  
  ggplot(aes(x = .fitted, y = Employed)) +  
  geom_point(color = "blue") +  
  geom_abline(slope = 1, intercept = 0, linetype = 2) +  
  labs(  
    title = "Actual vs. Fitted Values: Drift Model",  
    x = "Fitted values",  
    y = "Actual values"  
  )
```



Are the points close to the identity line?

Many points are very close to the identity line. There are outliers especially at the low and middle level of employment. At the low level of employment there more outliers above the interception line. At the mid level the outliers tend to be under the interception line. The high level of employment does not seem to have outliers. This seems to be because there are in general not many data points with high level of employment.

Are they systematically too high or too low?

There is no constant overestimation or underestimation across the whole plot, but the model shows small errors at the low- and mid level employment.

If yes, where does it come from?

Not applicable

2.2.5 Residual Diagnostics

The residuals of the drift model are inspected using `gg_tsresiduals()`. A good forecasting model should have residuals with no clear autocorrelation and a mean close to zero.

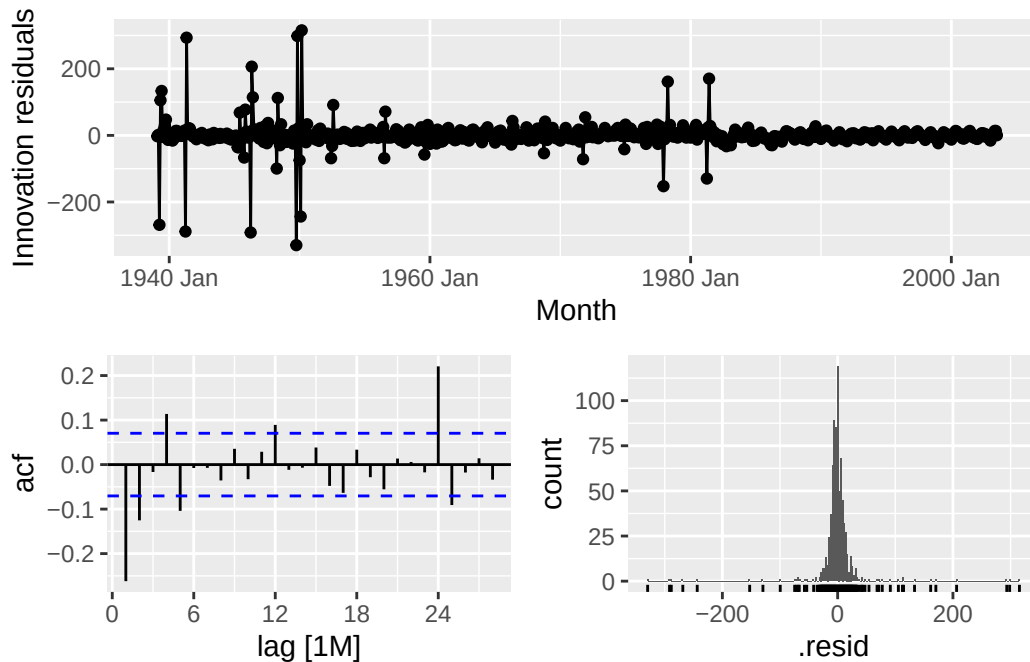
```
model_drift %>%  
  gg_tsresiduals()
```

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_point()``).

Warning: Removed 1 row containing non-finite outside the scale range (``stat_bin()``).

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_rug()``).



Are the residuals auto-correlated? How do you decide that based on the plots?

Yes, the residuals are auto-correlated. The ACF plot shows several spikes clearly outside the blue significance bounds (at the first lags and at lag 12 and 24 and etc.), which is more than the ~5% of spikes we would tolerate by chance for a white-noise series.

Do the residuals have zero mean? How do you decide that based on the plots?

Yes, the residuals appear to be centred around zero: the histogram is concentrated around zero and the residual time plot fluctuates around zero. Some large outliers are visible, but they do not shift the centre noticeably.

What do these results tell you about your model?

The autocorrelation in the residuals shows that the drift method does not capture all the structure (seasonality & short-term dependence) present in the data. There is still information left in the residuals that a better model could exploit.

2.2.6 Double-check Residual Diagnostics

The residual diagnostics are double-checked using the Ljung-Box test for residual autocorrelation and by calculating the mean of the residuals. Since the data is monthly and the seasonal period is 12 months, a lag of 24 is used for the Ljung-Box test.

```
# Ljung-Box test for residual autocorrelation
lb_drift <- augmented_drift %>%
  features(.resid, ljung_box, lag = 24, dof = 0)
```

```
lb_drift
```

```
# A tibble: 1 x 4
  Series_ID      .model lb_stat lb_pvalue
  <chr>         <chr>    <dbl>    <dbl>
1 CEU1000000001 drift      144.        0
```

```
# Calculate residual mean
residual_mean_drift <- mean(augmented_drift$.resid, na.rm = TRUE)

residual_mean_drift
```

```
[1] 2.467684e-14
```

Does the Ljung-Box test support your conclusions from the residual plots? How do you conclude that from the test result?

Yes, the Ljung-Box test supports the conclusion that the residuals are autocorrelated. The p-value is effectively 0, which is below 0.05, so the null hypothesis of no residual autocorrelation is rejected. This confirms what we saw in the ACF plot.

Does the residual mean support your conclusions from the residual plots?

Yes. The residual mean is closely to 0, so it supports the conclusion that the residuals are centred around zero.

2.2.7 Evaluate Point Forecast Accuracy of 2nd Benchmark Model

The trained drift model is used to forecast the remaining 20% of the time series. The forecast horizon is set to `test_length`, which is the number of observations not included in the training set.

```
# Create forecast for the same length as the test period
fc_drift <- model_drift %>%
  forecast(h = test_length)

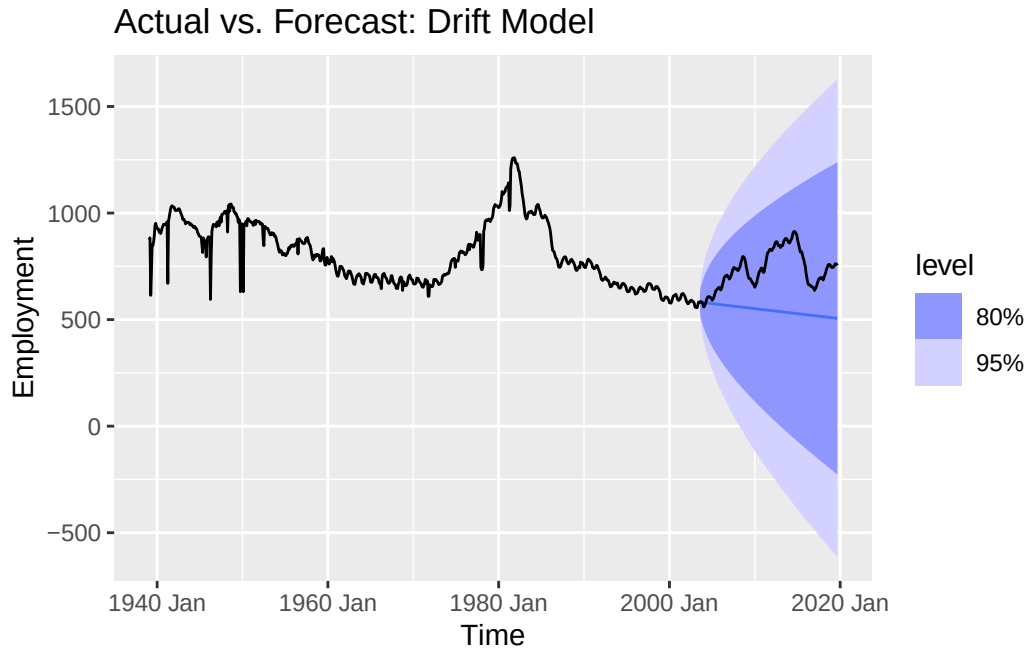
# Inspect forecast
fc_drift
```

```
# A tibble: 194 x 5 [1M]
# Key:   Series_ID, .model [1]
  Series_ID      .model      Month
  <chr>         <chr>      <mth>
1 CEU1000000001 drift    2003 Aug
2 CEU1000000001 drift    2003 Sep
3 CEU1000000001 drift    2003 Oct
4 CEU1000000001 drift    2003 Nov
5 CEU1000000001 drift    2003 Dec
6 CEU1000000001 drift    2004 Jan
7 CEU1000000001 drift    2004 Feb
8 CEU1000000001 drift    2004 Mar
9 CEU1000000001 drift    2004 Apr
10 CEU1000000001 drift    2004 May
# i 184 more rows
# i 2 more variables: Employed <dbl>, .mean <dbl>
```

2.2.8 Actual vs. Forecast Time Plot

The forecast values are plotted together with the original time series to visually compare the predicted values with the actual observations in the test period.

```
fc_drift %>%
  autoplot(mySeries) +
  labs(
    title = "Actual vs. Forecast: Drift Model",
    x = "Time",
    y = "Employment"
  )
```



2.2.9 Forecast Accuracy Metrics

The point forecast accuracy is evaluated using accuracy metrics. The two main metrics interpreted here are RMSE and MAPE.

```
accuracy_drift <- fc_drift %>%
  accuracy(mySeries)

accuracy_drift
```

```
# A tibble: 1 x 11
  .model Series_ID .type ME RMSE MAE MPE MAPE MASE RMSSE ACF1
  <chr> <chr> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 drift CEU1000000001 Test 191. 216. 191. 24.7 24.8 4.39 3.17 0.986
```

Actual vs. Forecast interpretation:

The drift forecast does not capture the development in the test period. The forecast is a straight line with a slightly downward slope, because the drift coefficient is the average change between the first and the last observation of the training set, and over the training set the series declines overall. In the test period, however, the actual employment rises again, so the

gap between forecast and actual values widens over the horizon. The actual values stay within the 80% prediction interval, but the point forecast misses the upward trend.

RMSE interpretation:

The RMSE of the drift model is 215.80. Since the RMSE is scale-dependent, it must be judged against the size of the values in the series: in the test period the actual employment mean is around 734, so an error of about 216 is roughly 30% of the typical value, which is large. This large error happens because the drift forecast slopes downward while the actual values rise, so the gap between forecast and reality keeps growing the further ahead we forecast.

MAPE interpretation:

The MAPE of the drift model is 24.78%. This means that, on average, the forecast error is about 24.78% of the actual employment level. This is high for a point forecast and confirms that the drift method is not accurate for this test period. (Note: the mean error ME is +190.73 and the MAE is 191.29 — both positive and almost identical, which shows the model systematically under-forecasts across the whole test period.)

2.2.10 Evaluate Point Forecast Uncertainty of 2nd Benchmark Model

The forecast uncertainty is evaluated by checking whether the residuals satisfy the assumptions needed for reliable prediction intervals. The same residual diagnostics as before are used, but the focus is now on constant variance and normality.

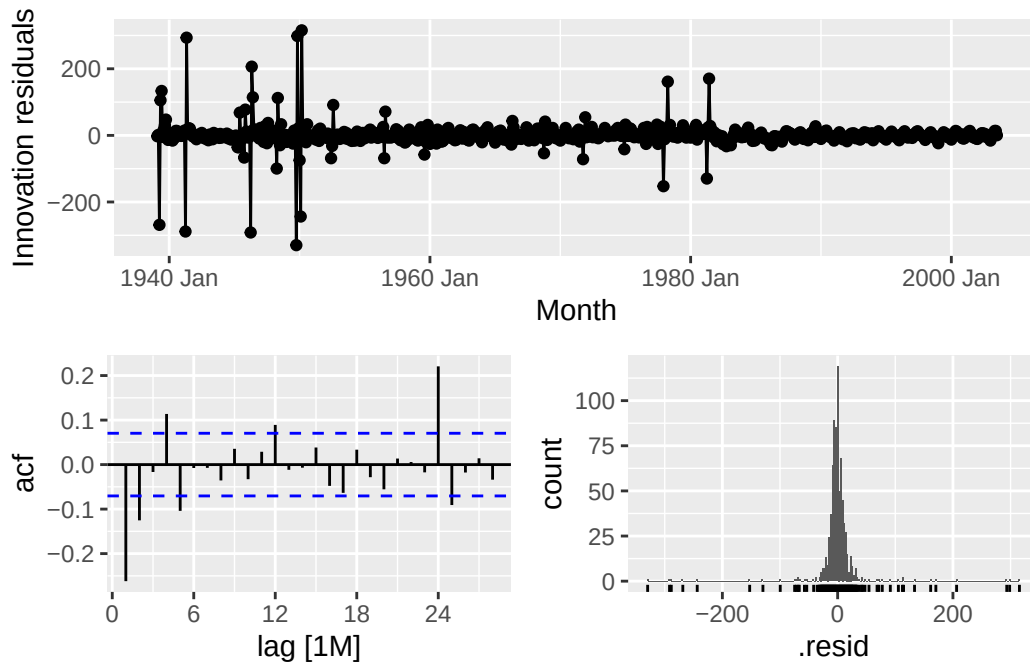
```
model_drift %>%  
  gg_tsresiduals()
```

```
Warning: Removed 1 row containing missing values or values outside the scale range  
(`geom_line()`).
```

```
Warning: Removed 1 row containing missing values or values outside the scale range  
(`geom_point()`).
```

```
Warning: Removed 1 row containing non-finite outside the scale range  
(`stat_bin()`).
```

```
Warning: Removed 1 row containing missing values or values outside the scale range  
(`geom_rug()`).
```



Are the residuals homoscedastic?

No, the residuals do not have constant variance. The residual time plot shows periods with much larger fluctuations than others, so the variance is not stable over time (property 3 is violated).

Are the residuals normally distributed?

Only approximately. The histogram is roughly symmetric and bell-shaped around zero, but it has heavy tails with several distinct outliers on both sides, so the normality assumption (property 4) is questionable.

2.2.11 Prediction Interval Boundaries

The prediction interval boundaries are displayed using `hilo()`. This shows the lower and upper bounds of the 80% and 95% prediction intervals for each forecasted month.

```
fc_drift %>%
  hilo(level = c(80, 95)) %>%
  select(Month, .mean, `80%`, `95%`)
```

```
# A tsibble: 194 x 4 [1M]
  Month .mean      `80%`      `95%`
  <mtb> <dbl>    <hilo>    <hilo>
1 2003 Aug  582. [534.5567, 628.6603]80 [509.6490, 653.5680]95
```



```

2 2003 Sep 581. [514.6328, 647.8013]80 [479.3853, 683.0488]95
3 2003 Oct 581. [499.2244, 662.4268]80 [456.0273, 705.6238]95
4 2003 Nov 580. [486.1486, 674.7197]80 [436.2368, 724.6314]95
5 2003 Dec 580. [474.5605, 685.5248]80 [418.7216, 741.3637]95
6 2004 Jan 580. [464.0271, 695.2752]80 [402.8194, 756.4829]95
7 2004 Feb 579. [454.2914, 704.2280]80 [388.1372, 770.3822]95
8 2004 Mar 579. [445.1860, 712.5505]80 [374.4189, 783.3176]95
9 2004 Apr 578. [436.5947, 720.3588]80 [361.4869, 795.4666]95
10 2004 May 578. [428.4330, 727.7376]80 [349.2118, 806.9587]95
# i 184 more rows

```

Does it make sense to include these prediction intervals in the model evaluation?

No, not as a reliable measure. The prediction intervals should be interpreted only with strong caution, because the residuals are autocorrelated, not homoscedastic, and only approximately normal. Since the interval width is computed assuming uncorrelated residuals with constant variance (and, by default, normality), these violations mean the intervals are not trustworthy and should not be used to compare the two models.

2.3 Comparison: Seasonal Naive Method vs. Drift Method

2.3.1 Comparison of the Results

The two models cannot be separated by their model fit: the residual diagnostics came out badly for both so both fail to capture all the structure in the data and we cannot tell from the fit which one is better. For the same reason the prediction intervals of neither model are reliable, so they cannot be used for the comparison either.

The decision therefore rests on point-forecast accuracy on the test set. Here the seasonal naive method is clearly more accurate than the drift method:

Metric	Seasonal Naive	Drift
RMSE	183.68	215.80
MAPE	20.78%	24.78%

The seasonal naive method has the lower error on both metrics. This is consistent with the time plots: the drift forecast is a straight line sloping slightly downward, which moves away from the rising actual values, while the seasonal naive forecast at least repeats the yearly seasonal pattern of the data.

2.3.2 Model of Choice: Seasonal Naive Method

The seasonal naive method has the lower RMSE and the lower MAPE on the test set. Since both models have an equally poor model fit and unreliable prediction intervals, point-forecast accuracy is the deciding criterion. But it should be noted that neither model captures the data well. Among the two, seasonal naive forecasts the test period slightly more accurately.